Results. As seen in Figure 6, the Belief State transformer outperforms the FIM model. The Belief State Transformer outputs consistently have correct grammar, whereas the FIM models often abruptly end in the middle of sentences. This grammatical flaw is consistently picked up by the GPT4 judge, and results in the FIM's lower winrate. See below for an example.

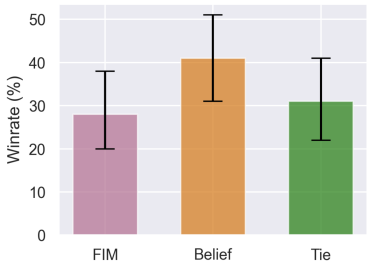


Figure 6: Unconditioned winrates.

Prefix: *You don't have to be scared of the loud dog, I'll protect you". The mole felt so safe with the little girl. She was very kind and the mole soon came to trust her. He was*

Belief: so gentle and kind. The little girl was so happy to have a new friend. She wanted to show the mole all her favorite things. She took him to the park and showed him all the different things she could see. The mole was so excited to see the different things. He ran around and played with the other kids. He even found a ball and a toy car. The little girl was so happy to see the mole having fun.

FIM: always there to protect her. One day, the mole was walking through the woods when he heard a loud noise. He looked up and saw a big, scary dog. The dog was barking and growling at the mole. The mole was so scared that he ran away as fast as he could. The dog chased after the mole, but the mole was too fast. The dog was getting closer and clos

Suffix: ∅

5.3 Ablations

Finally, we conduct a few ablations to verify our design choices. First, the **Belief Beamsearch** ablation uses beamsearch to generate the candidate set rather than the priority queue scheme in Algorithm 1. Next, in the unconditioned setting, the **Belief Next Score** ablation uses the next head T_n to score the suffix instead of T_p.

Figure 7 shows the performance drops in both cases. The **Belief Beamsearch** outputs is more prone to repetition than Algorithm 1. Next, the **Belief Next Score** ablation frequently outputs stories that end abruptly and incorrectly, similar to the FIM baseline in the unconditional setting.

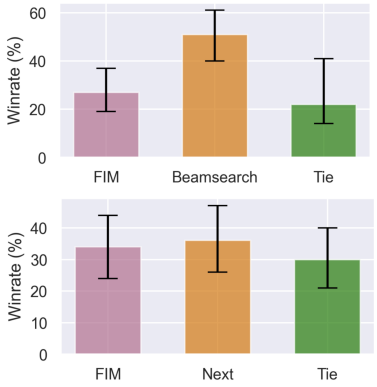


Figure 7: Ablations.

5.4 Understanding Belief States

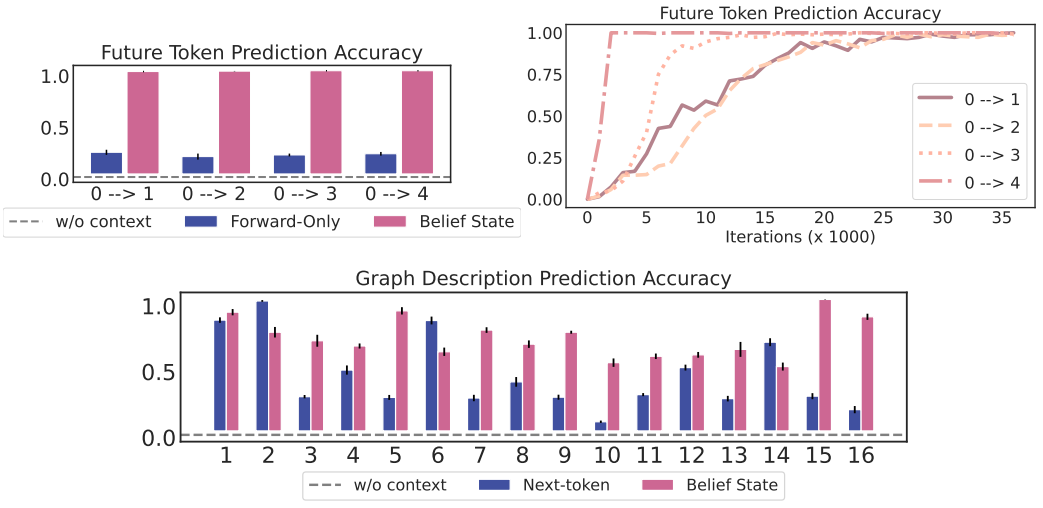


Figure 8: **Probing results on star graphs** show that Belief State Transformer's learned representation captures more information about future tokens (top-left). Top right: this belief state develops earlier in training for tokens which are further in the future. Bottom: besides future tokens, the inputted graph descriptions are also captured in the embeddings.

We investigated the representations learned by the Belief State Transformer to provide insight into what the states capture and how they are learned. First, we verified that the representations are indeed belief states, by training small MLP networks from the first hidden state on the G(2,5) Stargraph to predict the future token on a specific timestep. We found that the Belief State Transformer has the information to predict all future tokens, while the information contained within the state learned using the Forward-Only objective is incomplete (Figure 8, top-left).

An analysis of how the probe accuracy improved over the course of training revealed a clear trend. The belief state captures information about the tokens at the end of the sequence at the beginning of training, while information about the earlier tokens is learned later (Figure 8, top-right).

The same probing technique can understand how well the Belief State successfully captures information about the graph description. We found that the Belief State Transformer captures more information about the graph description than the Forward-Only model (Figure 8, bottom). Capturing the graph description completely is sufficient but not necessary for learning the belief state, since all the information in the graph description might not be used for predicting the future tokens.

6 Practical Implementation

Several alterations to the BST architecture and objectives support greater flexibility and scalability in training. Note that experimental results above Sections 3.5 and 5 were obtained using the original BST architecture and objective. We describe these changes below:

Subsampled loss: Aggressively subsampling prefixâĂŞsuffix pairs each iteration instead of using all $O(T^2)$ combinations achieves a substantial training speedâĂŘup with only minimal impact on final performance.

Decoupled text heads and Flexible loss weighting: We included a pure GPT-style next-token predictor T_{GPT} which takes in the forward embedding, i.e. $\mathsf{F}(\cdot)$, to predict the next token. We then included loss weights γ and λ for the pure next-token prediction and the BST objectives respectively:

$$\mathbb{E}_{t,x_{1:t},k\leq T-t}\left[\gamma \log \frac{1}{\mathsf{T}_{\text{GPT}}(x_{t+1} \mid \mathsf{F}(x_{1:t}))} \;+\; (1-\gamma)\right. \tag{9}$$
$$\left.\left(\lambda \log \frac{1}{\mathsf{T}_n(x_{t+1} \mid \mathsf{F}(x_{1:t}), \mathsf{B}(x_{t+k:T}))} \;+\; (1-\lambda) \log \frac{1}{\mathsf{T}_p(x_{t+k-1} \mid \mathsf{F}(x_{1:t}), \mathsf{B}(x_{t+k:T})}\right)\right]$$

Sharing encoder parameters We tie the forward and backward encoders by sharing all their parameters. Inspired by BERT (Devlin et al., 2018), we introduce a learned segment-embedding layer: it maps a binary segment ID (0 = prefix, 1 = suffix) to an embedding vector, which we add to the positional embeddings in each transformer residual block.

Table 1 presents a comparison of performance and training speed on TinyStories across the standard GPT, the original BST, and the improved BST which incorporates the above-mentioned modifications. As shown, by shifting the training objectiveâĂŹs emphasis toward next-token prediction (i.e., $\lambda = 0.9$), the improved BST model is able to match the standard GPT on next-token prediction. This is arguably important, since inference is usually carried out in a forward autoregressive fashion. Moreover, we accelerate BSTâĂŹs training by reducing its parameter footprint through shared encoders and by applying more aggressive subsampling of training pairs.

7 Other related work

Many previous works explore non-left-to-right approaches for sequence modelling. ELMo (Peters et al., 2018) and BERT (Devlin et al., 2018) were prominent approaches that trained language representations with past and future information. Gu et al. (2019) model the generation order as latent variables and performs beam search over the generation order itself to produce the final text. This is beneficial because in general, the best ordering for decoding is task-dependent. On the other hand, Welleck et al. (2019) explore generating text in a binary tree order, but their model struggles to outperform traditional left-to-right autoregressive generation in tasks like language modeling, machine translation, sentence completion, and word reordering. In Nguyen et al. (2024),

	GPT	BST (original)	BST (improved)
No. of encoder parameters during training	57 million	114 million	57 million
Next token loss	0.929	1.013	0.926
Previous token loss	-	1.110	1.268
Training speed (iterations/second)	~ 3.086	~ 0.205	~ 0.801

Table 1: Comparison of performance and training speed on TinyStories between standard GPT, the original BST, and the improved BST, measured on a single H100 GPU. The original BST trains on all $O(T^2)$ prefix-suffix pairs and uses separate encoders (see Equation (1) and Equation (2)). The improved BST subsamples 2% of all pairs using Equation (9) with $\gamma = 0.9$ and $\lambda = 0.0$, and uses shared encoders.

they propose training two separate transformers and implement several strategies to "meet in the middle" during decoding. In contrast, our approach involves jointly training a transformer capable of both forward and backward decoding, which has important implications for creating compact belief states. A general study of the quality of backward prediction is done in Papadopoulos et al. (2024) discovering that backward prediction is possible but generally slightly worse than forwards prediction. Somewhat further afield, combined forward/backward decoding approaches with RNNs have proved useful (Mou et al., 2015; Serdyuk et al., 2018; Mangal et al., 2019) and similarly for neural machine translation (Liu et al., 2016; Zhang et al., 2018).

The concept of predicting multiple tokens has been explored in other works. Gloeckle et al. (2024); Cai et al. (2024) aim to increase decoding speed by employing models that predict multiple tokens rather than just the next token. DeepSeek-V3 (Liu et al., 2024) pretrains the LLM to predict the next two tokens, and shows the objective is beneficial in the large scale setting. Such approaches align with our theoretical insights that predicting multiple tokens fosters the creation of more robust representations of future contexts.

Shai et al. (2024) show that transformers create a non-compact representation of belief states within their residual stream. Compact belief states are accessible in state-space models (Hasani et al., 2020; Gu & Dao, 2023), although this comes with different trade-offs compared to transformer-based approaches. For example, the Mamba training process is known to fail on star graphs (Bachmann & Nagarajan, 2024).

8 Conclusion

The Belief State Transformer advances goal-conditioned next-token predictors, effectively addressing the limitations of traditional forward-only transformers. The ability of our model to learn a compact belief state provides a maximal solution: since the belief state contains *all* the information useful in predicting the future, no more complex objective can elicit more information. Through experiments with both star graphs and story writing tasks, we have demonstrated the necessity of each component of our model, particularly in challenging scenarios where standard transformers struggle.

Looking ahead, while our results demonstrate the superior performance of the Belief State Transformer in small scale story writing, exploring its application to other goal-conditioned tasks would be valuable. Our current experiments serve as proof-of-concept. Further investigation into the scalability of our approach to larger practical scenarios is essential.

9 ACKNOWLEDGEMENTS

This work was supported in part by the NSF CAREER 2239301, NSF 2331783, and DARPA TIAMAT HR00112490421 grants. The authors would like to thank the Marco Rossi and the MSR AI Frontiers team for fruitful discussions and support, and the ICLR reviewers for their constructive feedback.

REFERENCES

Emmanuel Abbe and Enric Boix-Adsera. On the non-universality of deep learning: quantifying the cost of symmetry. *Advances in Neural Information Processing Systems*, 35:17188–17201, 2022.

Emmanuel Abbe and Colin Sandon. Polynomial-time universality and limitations of deep learning. *Communications on Pure and Applied Mathematics*, 76(11):3493–3549, 2023. doi: https://doi.org/10.1002/cpa.22121. URL https://onlinelibrary.wiley.com/doi/abs/10.1002/cpa.22121.

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Gregor Bachmann and Vaishnavh Nagarajan. The pitfalls of next-token prediction. In *Forty-first International Conference on Machine Learning*, 2024.

Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022.

Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

Ronen Eldan and Yuanzhi Li. Tinystories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.

Arvid Frydenlund. The mystery of the pathological path-star task for language models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 12493–12516, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.695. URL https://aclanthology.org/2024.emnlp-main.695/.

Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.

Alex Graves. Sequence transduction with recurrent neural networks. *arXiv preprint arXiv:1211.3711*, 2012.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Jiatao Gu, Qi Liu, and Kyunghyun Cho. Insertion-based decoding with automatically inferred generation order. *Transactions of the Association for Computational Linguistics*, 7:661–676, 2019.

Ramin Hasani, Mathias Lechner, Alexander Amini, Daniela Rus, and Radu Grosu. Liquid time-constant networks, 2020. URL https://arxiv.org/abs/2006.04439.